1 Wprowadzenie

Dzisiaj jest ten dzień, gdy rozpoczynamy nowe zajęcia. Jeśli dotarliśmy do tego momentu, to oznacza, że kwestie organizacyjne mamy już za sobą. Czy na pewno?

W trakcie tego semestru, jak zawsze, będziemy pracować w pewnym rygorze czasowym, ale dodatkowo będziemy musieli przestrzegać kilku zasad, aby prawidłowo oddać pracę do oceny.

Ten dokument (jak i każdy kolejny w trakcie semestru) w formie wynikowej (PDF) należy wgrać do ISOD-a przed końcem zajęć.

Dodatkowo cały kod (w formie źródłowej, bez zbędnych zależności) należy umieścić w wydziałowym repozytorium Git-a, dostępnym z ISOD-a.

Wynikowe zmiany, zgodnie z poleceniem przy każdym ćwiczeniu, należy oznaczać konkretnymi tagami (ze znacznikiem czasowym).

Brzmi groźnie? To nic wielkiego. Jest kilka poleceń, których musimy się nauczyć. A w razie problemów z pamięcią będziemy mogli sięgnąć do tego dokumentu.

Czy to zawsze zadziała? Absolutnie nie! Jeśli bardzo namieszamy, to te podstawowe polecenia nam nie wystarczą.

I co wtedy? Wtedy ktoś musi schować się w budce telefonicznej, założyć niebiesko-czerwony kostium i ruszyć z pomocą. Miejmy jednak nadzieję, że unikniemy tych sytuacji. Wystarczy, że...

1.1 Fantastycznie podstawowe polecenia Git-a i jak je znaleźć

Poniżej mamy spisanych kilka kroków, które są niezbędne, żeby nasza praca przebiegła pomyślnie i bez większych problemów.

1.1.1 Klonowanie repozytorium

W ISOD-zie, w miejscu z ocenami do naszych zajęć laboratoryjnych, znajdziemy niebieski przycisk *Przejdź do Projektora* (rys. 1).

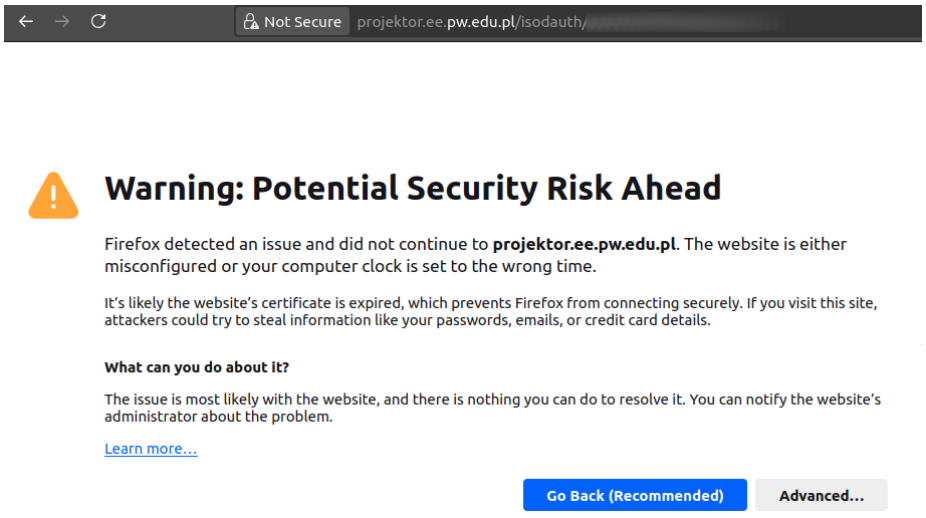
Po kliknięciu (*Przejdź do Projektora*) w przeglądarce powinna otworzyć się oczekiwana strona. Niestety ze względów niezależnych od nas (Wydziału) to połączenie jest szyfrowane, ale nie ma aktualnego certyfikatu. Dlatego, zamiast oczekiwanej strony *Projektora*, wyświetli się strona informująca nas o zagrożeniu (rys. 2) i ryzyku, które podejmujemy, ignorując to ostrzeżenie (rys. 3).

Jeśli to, co powyżej, mamy już za sobą, to teraz będzie już tylko trudniej.

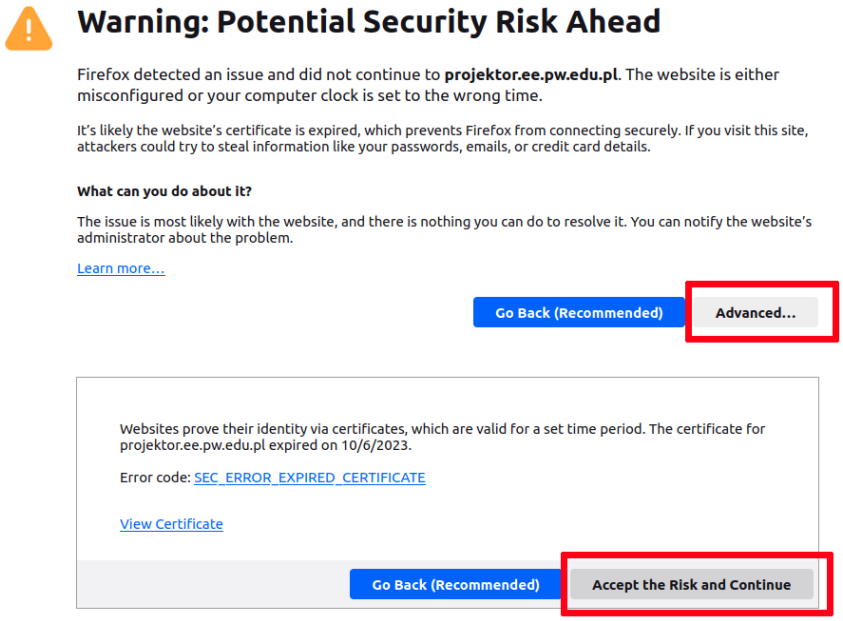
Na nowo otwartej stronie próbujemy odszukać link zaczynający się od `https://` (rys. 4). (Uwaga dla chętnych: można też wybrać wersję z `ssh`, ale to będzie wymagało podania klucza publicznego. Projektor ogranicza nas do jednego klucza, czyli jednej maszyny.) Klikamy „w niego”, co skutkuje otwarciem strony *401 Unauthorized*. Ignorujemy (domyślny) formularz logowania, a następnie z adresu przeglądarki kopiujemy zawartość (cały adres).



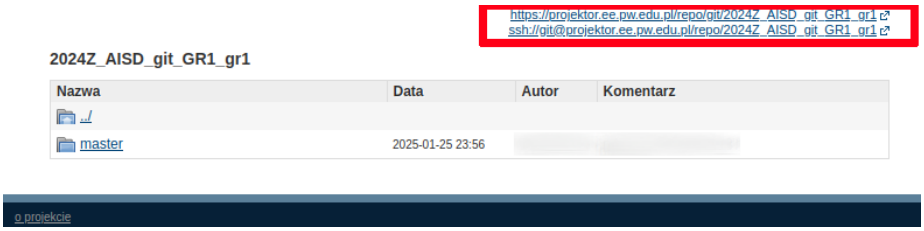
Rysunek 1: Gdzie znaleźć przycisk *Przejdź do Projektora*?



Rysunek 2: Ostrzeżenie o ryzyku ze względu na brak certyfikatu.



Rysunek 3: Zaakceptowanie ryzyka braku aktualnego certyfikatu.



Rysunek 4: Widok *Projektora* ze wskazaniem adresów do klonowania repozytorium.

Naszym kolejnym krokiem jest wybranie lokalizacji na dysku, gdzie będziemy chcieli sklonować repozytorium. Otwieramy w tym miejscu terminal (w przypadku MS Windows szczerze polecam konsolę *GitBash*). Gdy już jesteśmy ustawieni w docelowym miejscu wykonujemy:

```
git clone -c http.sslVerify=false <skopiowany_adres>
```

Warto zwrócić uwagę na flagę `-c http.sslVerify=false` – dodajemy ją, ponieważ musimy jawnie zadeklarować, że godzimy się z ryzykiem. (Uwaga dla chętnych: można tę flagę ustawić na stałe, wykonując:
`git config http.sslVerify false`.)

Jeśli wszystko przebiegło poprawnie, to teraz mamy już sklonowane repozytorium.

Wykonujemy zadania, implementujemy kod i... wysyłamy zmiany do repozytorium.

Nim ten proces nam się uda, musimy jeszcze dokończyć konfigurację aktualnego repozytorium, w związku z tym ustawiamy nazwę użytkownika (polecam swoje imię i nazwisko) oraz adres e-mail:

```
git config user.name "Jan␣Kowalski"
git config user.email "jan.kowalski.stud@pw.edu.pl
```

Teraz już możemy wykonać standardowy ciąg czynności z zapisywaniem naszej pracy. Co kolejno robimy?

1. Sprawdzamy status poleceniem `git status`.
2. Dodajemy pliki do najnowszej zmiany (commit-a): `git add nazwa_pliku`.
3. Po dodaniu wszystkich plików zapisujemy commit-a:
   `git commit -m "Nasz kreatywny komentarz"`.
4. Dodajemy tag ze znacznikiem czasowym:
   `git tag -a LAB_0_FINAL -m "Koniec lab 0."`.
5. Ściągamy ewentualne zmiany z repozytorium zdalnego (powinniśmy od tego zacząć zajęcia!): `git pull`.
6. Wysyłamy nasze zmiany do głównej gałęzi (`master`):
   `git push origin master`.
7. Wysyłamy nasz tag (w tym przypadku: `LAB_0_FINAL`):
   `git push origin LAB_0_FINAL`.
8. Dla pewności sprawdzamy, czy wszystko się udało: `git status`.

2 Zadania

Skoro już wiemy, jak będziemy pracować z repozytorium, to czas wykonać jakieś zadanie.

2.1 Zadanie 1.

W otrzymanym projekcie musimy uzupełnić implementacje dwóch klas w plikach:

- `LinearSearch.java`;
- `BinarySearch.java`.

Obydwie klasy mają wykonywać wyszukiwanie wartości (w tablicy liczb całkowitych) zgodnie z opisem „narysowanym" na tablicy. Gdy poszukiwana liczba nie występuje w przeszukiwanej tablicy, należy zwrócić `-1`.

2.2 Zadanie 2.

Proszę zweryfikować poprawność otrzymanych i wykonanych implementacji algorytmów poprzez napisanie testów jednostkowych. Każda klasa implementująca algorytm powinna być przetestowana.

2.3 Do przećwiczenia

To było proste ćwiczenie, prawda? Spróbujmy jeszcze przez chwilę pozostać w problematyce wyszukiwania binarnego.

Poniżej mamy dwa zadania, z którymi warto się zmierzyć, aby utrwalić (i poszerzyć) zdobytą wiedzę.

Rozwiązania należy umieścić w repozytorium i oznaczyć tagiem `LAB_0_EXTRA`.

2.3.1 Zadanie 3.

Sprawdźmy plik z klasą `MountainPeakProblem`. W komentarzu jest opis problemu.

2.3.2 Zadanie 4.

Sprawdźmy plik z klasą `HotOrColdProblem`. W komentarzu jest opis problemu.